TECHNICAL REPORT / CPU BACKENDS

Compiling scheduled egglog rules into reusable CPU execution blocks

An architecture proposal for egglog and eggcc, evaluated on frozen sum-bits and Raytrace Pass 3 workloads.

Eggfuse compiles rule-local joins and effects into typed recipes, shares entry work across consumers, and carries checked references to committed constructor results across scheduled Runs. It leaves Free Join responsible for residual matches. The architectural proposal is to expose these operations as optional physical plans inside egglog, where the native engine can retain ownership of scheduling, merges and extraction.

Latest Pass 3	Paired old ms	Candidate ms	Fixed ratio	Paired ratio
sum-bits	30.290	16.379	2.056×	1.849×
raytrace	12,208.315	6,140.271	2.200×	1.988×

Latest verified checkpoint `502e29cd`. The control is the earlier Cell CPU engine. Fixed ratios use the predeclared reference times; paired ratios use the control rerun in the same session. Candidate builds include native CPU, LTO and held-out PGO. Section 5 gives the full method and data.

> **The adoption decision.** Prototype shared entry packets and checked register joins beneath egglog's existing evaluator. The pinned cross-engine audit has a concrete sum-bits endpoint discrepancy even though projected table counts agree. Treat the timing differences as architecture evidence. Require the native observable-result and extraction gates in §7 before enabling an alternative backend for production.

Performance checkpoint `502e29cd` · memory implementation `4d9ea942` · Apple M3 Max · Rust 1.94.0. Sections 5–6 report performance; Section 8 reports memory. Sample data, source identities and validation scripts are included.

1. Where the compiler meets the engine

Eggcc supplies a scheduled equality-saturation program and uses an extractor to choose a result for its next pass. Egglog supplies incremental relational evaluation and congruence closure. A useful extension point is the compiled rule: it already identifies variables, table calls, key columns, effect destinations and ruleset ownership. Eggfuse makes those relationships explicit for a standalone executor. eggcc source · egglog semantics

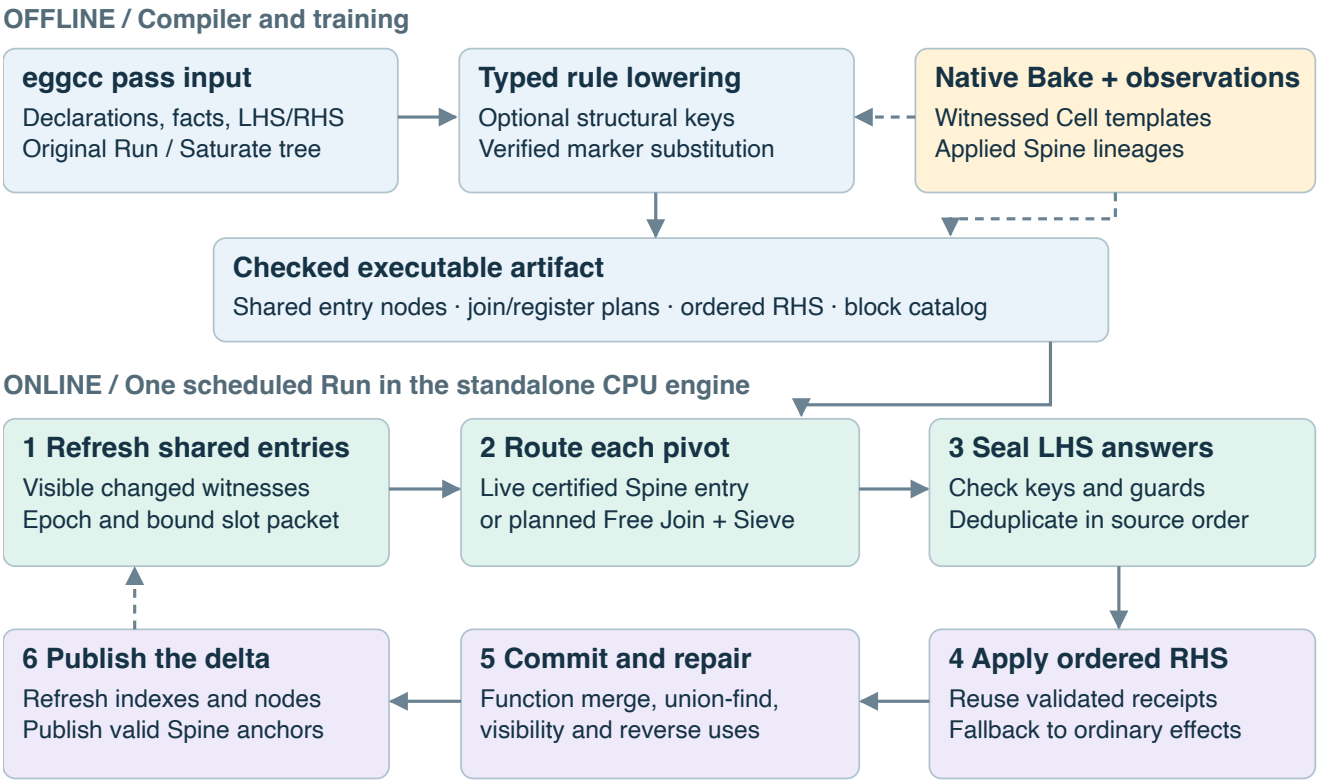


Fig. 1 The compiler produces a checked artifact. The colored runtime path repeats at the caller's next legal Run. Dashed inputs carry training evidence or refreshed state across a boundary. Native extraction sits outside the replay timer.

Two kinds of prior knowledge. The compiler proves facts such as "this key is bound" or "this RHS requests the same constructor as that producer." Training observes how often a path actually wins. The first supplies the legality conditions. The second decides whether spending memory and compilation effort on the path is worthwhile. Compiler entry · Artifact loader

What sharing can save. Suppose d changed rows satisfy one atom shape watched by m consumers. If all m separately decode and test that atom, its entry work is dmc, where c is the cost per decode and test. With a shared entry it is approximately $dc + d\sum_i p_i$, where p_i is the cost of projecting and delivering the packet to consumer i. Each consumer still pays for its residual join, sorting and effects. This local model predicts a saving only when dispatch and maintenance cost less than the redundant entry work. Egglog can already share physical indexes, so that common cost is outside this comparison.

2. From a rule to a reusable plan

SOURCE TRANSFORMATION

The Rust front end reads the flattened pass, installs explicitly requested structural-key selection, runs Bake on a compatible training input, selects markers, and lowers the transformed program to Recipe IR. A marker represents a positive conjunction with explicit ports. The compiler emits maintenance rules to populate it before dependent consumers run. Its selection problem trades saved consumer atoms against maintenance cost, with structural estimates or measured inputs. Observing a Cell during Bake proposes reuse. The source transformation still needs a rule-derived positive view and a legal place in the schedule. Marker selection · Cell witnesses

PHYSICAL LOWERING

A rule becomes typed query atoms, input slots, effect destinations and a source-owned control tree. The compiler enumerates legal pivot-specific orders. A register kernel can admit a connected query with one indexed expansion, subsequent full-key reads and primitives whose operands are bound. Fresh columns write registers. Repeated variables, literals and previously assigned columns generate equality checks. The loader reconstructs physical plans from the typed recipe and checks serialized plans. Kernel derivation · Kernel interpreter

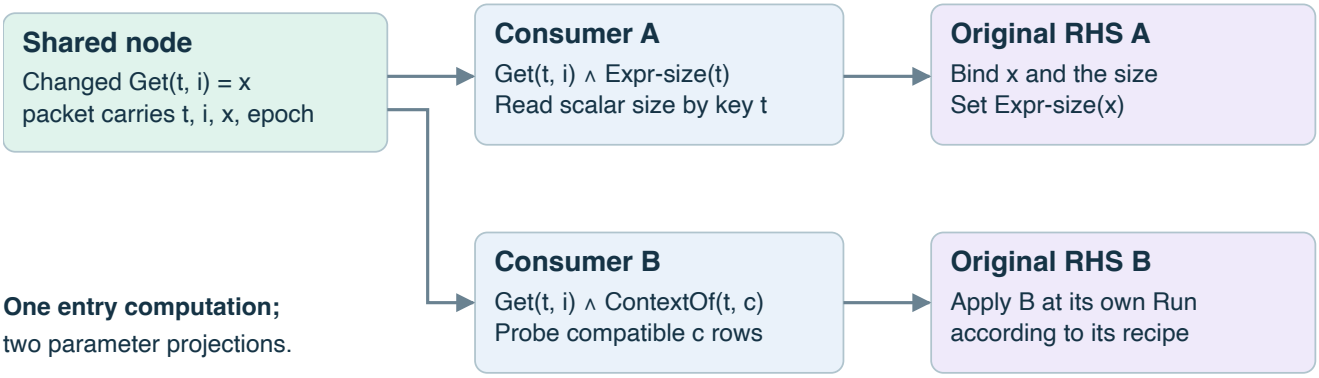


Fig. 2 Two consumers can subscribe to the same normalized atom shape. An arrival supplies a correlated row, its witness and epoch. Consumer projections map packet positions to local slots. An update to the other atom can also be the rule's pivot.

For a changed row r and consumer watermark w, entry selection restricts the delta to visible rows with $r.\text{epoch} \geq w$. The Sieve can prove that necessary domains do not intersect. Planned Free Join can probe compatible keys before materializing seeds and can start at a selective indexed domain. A fingerprint narrows the candidates. The matcher checks full values before emitting a binding. Ownership by epoch, witness and pivot plus source-slot deduplication eliminates duplicate derivations at the Run boundary. Shared nodes · Join routing · Sieve

3. Cells, repair and the Run boundary

A **Cell template** contains an ordered effect recipe. An **instance** records its template ID, bound slots, optional parent and live fact positions. A constructor fact can be named by a `FactRef {cell, op}`. The recipe tells the engine which relation and slot operands that position denotes. Decoding those operands produces the current logical row. A mutable function has a current key, value, owner and epoch. Query indexes and table widths still matter: storing facts through shared recipes trades duplicate representation for decoding, liveness tests and reference maintenance. Graph and repair · Frame storage

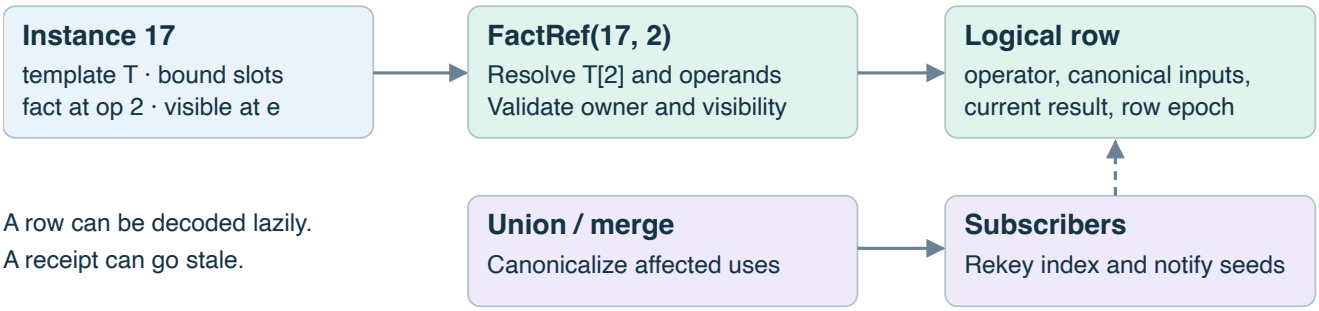


Fig. 3 The address of a constructor row combines a concrete Cell instance and an instruction position. A mutable function row has its own current owner. Purple shows the repair path after a key changes.

Physical object	What consumers may reuse	What can invalidate it
Cell fact position	A committed constructor witness	Deletion, subsumption, movement or a changed canonical key
Function receipt	The current committed function row	An ordered merge or replacement of its current value
Index bucket / packet	Candidate witnesses for a key and epoch	Membership, visibility or representative change
Spine cursor	Bounded producer lineage at a live anchor	A stale receipt or failed consumer precondition

EXECUTION CONTRACT

```
for each Run selected by the source schedule:
    read all rule LHSs against the current snapshot
    seal complete bindings in the rule's original input order
    apply their RHS instructions in order
    repair congruence and rekey affected rows
    publish visible changes, update subscribers and watermarks
    evaluate the next repeat/saturation condition
```

On union, reverse dependencies identify constructor and function keys involving an affected representative. The engine invalidates decoded payloads, handles rekeying or coalescence and delivers changes to the relevant indexes. Consumers must be able to observe old-key removal, new-key arrival, visibility and forwarding. A compiled block does not remove this repair obligation. A cheap RHS path can avoid registration of a temporary frame when a typed single function write can be completed directly. A failure of its preconditions resumes ordinary effects. Run and RHS execution · Storage and kernel ablation

4. How a block crosses Runs

A **Spine** follows committed facts that serve as anchors for subsequent rules. The compiler records which consumer pivot can read a producer's output and whether its remaining ports can be obtained exhaustively. The producer publishes a weak reference when its fact becomes visible. A later consumer can use the reference only at its own scheduled occurrence. The next matched rule can have other LHS entries, other matches and ordinary effects in the same Run.

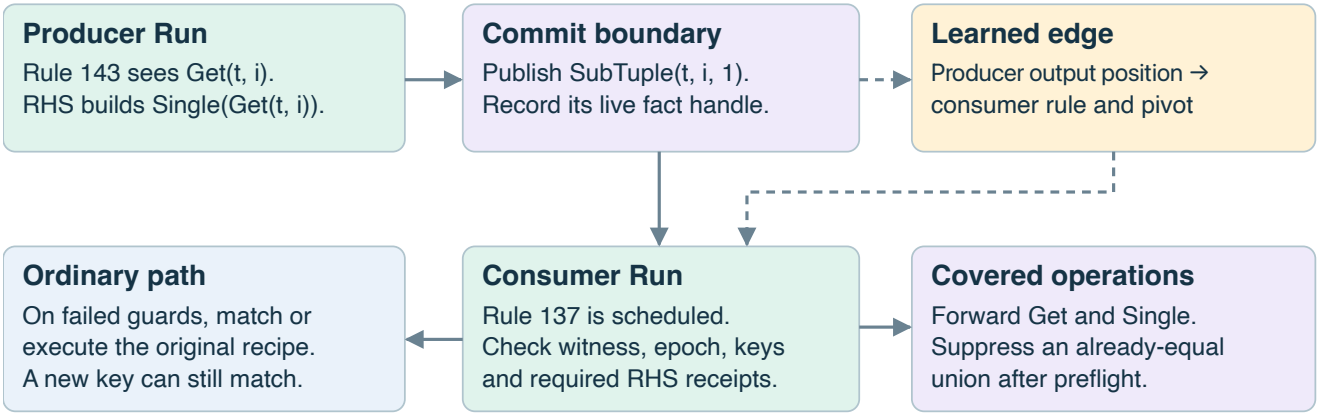


Fig. 4 An observed producer/consumer pair from Raytrace. The published SubTuple fact enables a later pivot. Producer equalities and constructor receipts can also satisfy part of the consumer RHS. The dashed edge requests a switch. The current runtime state decides whether it succeeds.

Concrete saving. Rule 143 witnesses `Get(t,i)`, constructs its `Single` and equates that result with `SubTuple(t,i,1)`. A later rule 137 requires the same constructor key. The compiler records the donor instruction, distance, and consumer destination. The runtime validates the current row and canonical keys before reusing it. A whole-RHS preflight checks every required receipt before any effect is committed. An unexpected replacement chooses the ordinary arm. This avoids failed speculation partially applying a write. Switch construction · Measured pair

Learning larger blocks. Start with 32 applied consumer/pivot leaves. A training round counts witnessed adjacent units on a winning lineage. A proposed binary unit names a left unit, the publishing instruction and a right unit. Freeze the catalog during each run, bound its size, and let an actually applied macro become a later operand. The donor used here has a 64-unit ceiling, at most four primitive steps, eight nodes, up to eight additions per round and a 512-entry candidate budget. When transferring, the harness asks the compiler to resolve each leaf's typed rule signature on the target. It remaps only reachable joins and rebuilds every RHS handoff against that target. Typed leaf transfer · Bounded composition

5. Latest verified performance checkpoint

502e29cd / FROZEN PASS 3 ACCEPTANCE

The latest complete fixed-target acceptance file in the inspected branch records implementation `502e29cd`. It compares two frozen Pass 3 programs with the original Cell executable at `c8ee1a3f`, whose binary SHA was verified. Each case retains 32 primitive block leaves and 32 learned joins. One warm-up per arm is excluded, followed by 11 measured sum-bits pairs and seven Raytrace pairs in fresh, alternating processes.

Pass 3	Paired old ms	Candidate ms	Fixed ratio	Paired ratio
sum-bits	30.290	16.379	2.056×	1.849×
raytrace	12,208.315	6,140.271	2.200×	1.988×

TWO DISTINCT ACCEPTANCE DENOMINATORS

The fixed ratio divides the previously declared original median by the new candidate median. The paired ratio divides the original median remeasured in the same session by that candidate median. Both use the original Cell engine as control. Section 6 defines the timed intervals and build settings.

Pass 3	Fixed reference ms	Fixed 2× cutoff ms	Candidates below cutoff
sum-bits	33.6750	16.8375	9/11
raytrace	13,508.0780	6,754.0390	7/7

Both candidate medians passed the fixed thresholds. The concurrent paired ratios were **1.849×** and **1.988×**. Thus the fixed-target checkpoint is complete, while a twofold improvement against both contemporaneous controls remains unestablished. The original executable ran faster in this session than in the session that defined the fixed thresholds.

CLI, MEMORY AND LOGICAL WORK

Pass 3	Old CLI ms	New CLI ms	Old RSS MiB	New RSS MiB
sum-bits	126.885	97.974	161.14	152.77
raytrace	15,360.573	8,493.103	2,907.45	1,978.20

Pass 3	Old visits	New visits	Pairs	Runs	Resumes / chain
sum-bits	18,730	18,693	11	1428	3,034 / 12
raytrace	6,916,788	6,798,165	7	16883	663,403 / 29

CLI time includes loading and exporting the same Cell graph. RSS is sampled after inference. Logical visits are the existing Cell work counter. Both independent full audits and forced-collision sum-bits matched the frozen RHS, effective-write, schedule, graph and Spine digests. The recorded test counts were 26 all-feature runtime, 12 ordinary runtime, nine minimal runtime and 15 compiler integration tests.

> **Build treatment.** The candidate uses fat LTO, one codegen unit, native CPU code generation and offline PGO trained solely on the other workload. LLVM's value-profile counter capacity is 16 during training. The measured binary collects no profile. These comparisons combine execution and build changes; they do not isolate PGO's contribution.

6. Benchmark method and interpretation

The performance evaluation covers the frozen sum-bits and Raytrace Pass 3 programs. Each executable runs the same source schedule on one CPU thread. Cell input consists of the checked recipe artifact, the marker representation and a fixed catalog of 32 primitive leaves plus 32 learned joins. The artifact, recipe, executable and opposite-workload profile hashes are bundled with the accepted measurements.

TIMED INTERVALS

Inference begins after artifact loading and runtime initialization and ends after the scheduled engine execution. It includes matching, RHS evaluation, rebuild and incremental publication. Complete CLI time also includes loading, diagnostics and graph export. Offline Bake and PGO training are outside these clocks. Each measured process starts fresh; one warm-up per arm is excluded, and control/candidate order alternates.

CONTROLS AND BUILD SETTINGS

The paired control is the original Cell executable at `c8ee1a3f`, verified by SHA-256. The candidate checkpoint is `502e29cd`. Its compiler configuration includes native CPU generation, fat LTO, one codegen unit and held-out PGO. The profile used for sum-bits comes from Raytrace; the profile used for Raytrace comes from sum-bits. These ratios measure the combined executable changes. A PGO-only claim requires a pair with every other build setting fixed.

FIXED TARGETS AND PAIRED RATIOS

The predeclared reference medians were 33.675 ms and 13,508.078 ms. Their twofold cutoffs are 16.8375 ms and 6,754.039 ms. The accepted candidate medians, 16.379 ms and 6,140.271 ms, pass both. The original remeasured in that session was 30.290 ms and 12,208.315 ms, yielding paired ratios of 1.849× and 1.988×. The measured pairs answer present-machine improvement; the fixed cutoffs answer progress against the declared target.

OBSERVATIONS THAT MUST AGREE

The candidate preserves the complete source Run sequence, ordered RHS inputs, effective write order, final logical graph and winning Spine lineage against the frozen Cell reference. The audit executable is independent of the timed build. Forced fingerprint and Sieve collisions exercise exact fallback checks. Counts of candidate visits, bindings, constructor requests and committed writes describe separate work units. A reduction in one count does not by itself establish a reduction in time.

MEMORY HAS ITS OWN MATCHED EXPERIMENT

Section 8 tests reader-lifetime reclamation and source-artifact release at ordinary release settings. Its control is `27f098a2` and its implementation is `4d9ea942`. The 1,432.68 MiB Raytrace peak belongs to that configuration; the performance checkpoint's 6.140 s and 1,978.20 MiB belong together. Each table reports its own treatment and sample counts.

> **Rebuild the performance tables.** `reproduce/render-performance-data.py` validates all measured samples and generates the tables from `evidence/performance-acceptance.json`. The archived full-suite inputs and results remain in the companion data package. The current PDF's performance tables use the two-workload checkpoint.

7. Semantic boundary and upstream decision

The current within-engine audits compare ordered RHS inputs, complete schedule boundaries, committed operation counts, final logical graphs and winning Spine lineages against frozen Cell references. Sum-bits has 1,428 audited Runs, 3,034 resumed steps and maximum depth 12. Raytrace has 16,883 Runs, 663,403 resumed steps and depth 29. These tests catch an illegal early activation or a dropped binding between Cell configurations. Cross-engine replacement also needs comparison against egglog.

> **A reproducible difference.** On sum-bits, both the original native export and current fixed-64 Cell export have 3,206 rows after projection to original table names; all per-table counts agree. Grounded reconstruction nevertheless finds a visible Cell `TermSingle` with no matching native key. All 19 constructor occurrences in the witness resolve to visible Cell rows. Appending the ground check to the original flat source gives `CheckError` in the timed baseline binary. The same check fails on the marked input in native egglog. The archive includes the concrete term, both exports and replay commands.

The diverging term can be written compactly with `g(i) = TermGet(TermArg(), i)` and `b(op,x,y) = TermBop(op(),x,y)`:

```
q = b(Div, g(1), g(2))
p = b(Mul, q, TermConst(Int(2)))
witness = TermSingle(b(Add, g(0), b(Sub, g(1), p)))
```

This locates an observable membership difference. It leaves open whether the changed graph produces a different Tiger-extracted program and which implementation decision first caused the divergence. In particular, an order-sensitive merge can depend on an earlier schedule or binding-order difference. The table below identifies a smaller upstream experiment that would distinguish these causes.

Proposed seam	Retain in egglog	Acceptance evidence
Compiled LHS entry	Native Run snapshot, watermark and action-buffer order	Shadow each pivot and compare complete substitutions, including misses and ties
Register / writer plans	Native value interning, conflict resolution and application order	Compare key reads and ordered effects for insert, replace and no-op
Receipt publication	Native rebuild, subsumption and canonicalization	Invalidate on old/new keys, visibility and forwarding; test deleted and merged witnesses
Cross-Run block	Native scheduling and extraction	Check guards against current rows. Compare grounded queries and extracted IR through all passes

The first implementation milestone is an opt-in alternative matcher feeding egglog's ordinary action buffer. Run the exact counterexample as a regression and capture the first divergent source Run. Only after equality partitions, function outputs, visibility and selected terms agree should a production trial replace native effect or storage handling. A final trial should include offline cost amortization, total compiler wall time, peak memory and execution of the resulting binaries.

8. Memory and cross-Run lifetime

6 NOVEMBER 2026 / TWO-WORKLOAD MEMORY STUDY

This follow-up measures the standalone Cell CPU engine at `27f098a2` and the lifetime changes implemented in `4d9ea942`. It uses the frozen sum-bits and Raytrace Pass 3 artifacts, their original 64-unit Bake catalog, one CPU thread and ordinary release builds. The memory matrix has its own control and ordinary release configuration; no PGO is used here.

Pass 3	Control ms	Reclaimed ms	Control MiB	Reclaimed MiB
sum-bits	19.158	18.740	152.18	146.81
raytrace	6,746.279	6,716.853	1,856.79	1,432.68

Raytrace's peak RSS fell by **22.8%**. In the same matrix, complete CLI medians were 10.373 and 10.374 s. The median of paired inference differences was +16.663 ms, with only one of four reclaimed runs faster. A lower ratio of medians therefore supplies little evidence of a latency improvement.

RECHECKING THE NATIVE MEMORY GAP

Two new runs of eggcc's pinned native egglog `ebba7bb` executed the original unmarked Raytrace source with a verified one-worker pool. macOS `time -l` recorded 462.375 and 450.047 MiB peak RSS; the median was 456.21 MiB. Native timing included parsing, initialization and inference, without a graph export. Cell RSS was sampled after inference and before its export, including its artifact loader and generated tables. The control/native memory ratio was approximately **4.07×**, falling to **3.14×** with reclamation. Source and binary hashes accompany the measurements.

COUNT LIVE ALLOCATIONS ONCE

A diagnostic allocator attributes requested bytes to the allocating operation and subtracts them on deallocation. A shared Rc allocation is charged once. Reallocation retains its original category. At Raytrace inference completion it counted **1,762.98 MiB** still live.

Allocation origin	Live requested MiB
graph	494.6
artifact	378.5
search	215.4
columns	163.4
entries	149.0
spine	121.3
other	77.0
row payload	76.9

The largest remaining categories include RHS (62.6 MiB), Sieve (22.1 MiB) and captured history (2.3 MiB). Allocation origin can differ from the eventual owner: matched parameters can become persistent Cell slots. Category peaks occur at different times. The ledger excludes allocator fragmentation and C allocations; its instrumented timing and RSS are excluded from the speed comparison.

Evidence: `evidence/memory-lifetimes.json`, rendered by `reproduce/render-memory-data.py`. Program identities and all three memory matrices are recorded in the evidence file.

Consumers determine retention

A shared entry node owns a set of declared rule consumers. After a source ruleset occurrence, each rule's logical watermark advances, including rules omitted by the ready set. The node can release a materialized packet once its epoch is **strictly below every consumer's watermark**. An equal epoch can still seed a match. A rule appearing in several sets advances with its latest actual occurrence.

The implementation keeps a compact membership header so join planning sees the same cardinality. A changed fact can rebuild its packet through the existing publication stream. An explicitly older watermark request marks the node cold and reconstructs its visible rows. Reclamation can run immediately or conservatively every 32 graph epochs. The latter delays release without advancing the readers.

On Raytrace, immediate reclamation reduced retained entry packets from **665,584** to **82,355**. Entry-origin requested storage fell from 149.0 to 72.4 MiB. There were 882,328 retirements over the schedule, including repeated incarnations of some witnesses.

COMPLETION RECEIPTS NEED FEWER PRODUCER OWNERS

The final Raytrace Cell graph has 1,034,514 logical rows. Generated `CellSpineView*` tables account for 213,334; `CellSpineApplied*` tables account for 121,502. Together they contribute **32.4**% of the rows. This table-name grouping describes the measurement. The optimization uses executable read contracts.

The three populated Applied tables have no declared query or RHS readers. Their Unit values use an old-value merge. The compiler derives origin demand from all rule queries, executable table reads and calls in merge expressions. It checks an emitted certificate against the target program. An unread receipt can keep its key, value and rebuild identity while releasing its optional creating-frame reference. An assertion or later declared rule that reads the table retains that producer.

Pass 3	Full live Cells	Embedded snapshots	Both forms
sum-bits	2,385 → 2,177	490 → 639	2,875 → 2,816
raytrace	601,924 → 485,196	85,989 → 163,541	687,913 → 648,737

Some remaining producers change from full Cells into embedded snapshots after an unnecessary second owner is removed. The combined count measures that distinction. Every completion-table row stays in the logical export. A caller choosing the new mode can receive `None` from the optional origin accessor for an unread receipt.

RELEASE VALIDATION IR AFTER INITIALIZATION

A teardown probe freed about 323.7 MiB by dropping the original Raytrace validation JSON, plus 17.1 MiB for its input buffer. The explicit `--release-source-artifact` option releases these after runtime plans are initialized. Typed plans, structural-key nodes and the original source SHA remain available. The residual object has a provenance-only schema, which the source loader rejects. Offline Bake or revalidation requires reloading the original artifact.

For sum-bits, the combined live-requested footprint fell from 131.99 to 22.08 MiB, while peak RSS changed by much less. Its parser/setup high-water had already occurred. This separates the benefit of reclaiming completed input state from the cost of initially loading it.

Cost, correctness and remaining scope

The primary Raytrace matrix isolates entry retirement from validation-IR release.

Arm	Inference s	Peak RSS MiB
Control	6.822	1,864.23
Consumed entries	6.813	1,788.11
Validation IR	6.816	1,549.02
Both	6.719	1,475.88

The final matrix combines all three changes, with 32-epoch retirement. It rotates eight processes per sum-bits arm and four per Raytrace arm. Compilation and diagnostic runs are outside the measurements.

Pass 3	Paired delta	Faster pairs	Control CLI ms	Reclaimed CLI ms
sum-bits	-0.362 ms	6/8	116.665	116.369
raytrace	+16.663 ms	1/4	10,372.988	10,374.331

The primary sum-bits retirement-only arm was slower in all eight pairs. The evidence supports a memory reduction with a small, workload-dependent maintenance cost. Both lifetime features remain opt-in.

AUDIT THE SAME SCHEDULED OBSERVATIONS

Full audits matched the frozen graph, schedule, ordered RHS inputs, effective writes and Spine trajectory. They include the combined mode and forced fingerprint/Sieve collisions. Twenty-eight runtime tests and fifteen compiler tests passed, covering reader lifetimes, backward watermark requests and rejected certificates.

These are within-Cell checks. The grounded cross-engine witness in §7 requires its own resolution. The memory comparison establishes within-Cell equivalence. Section 5 gives the independent fixed-target and contemporaneous paired performance checks.

WHAT CROSS-RUN COMPLETION CAN STILL ABSORB

History-reference trimming can recover only a few MiB in this measurement. Larger structures hold graph records, parameters, indexes and shared entry state. Future blocks should expose reader demand and last-use ports, then measure reconstruction costs against another retained row copy.

Generated view and completion rows are the next candidate. A compressed form must still support required canonical-key lookups, union forwarding, last-writer publication and exports. Their generated origin alone cannot justify removal.

> **Reproduce the update.** `reproduce/memory-replay.sh` accepts checkout `4d9ea942` and two frozen program files. Features `entry-retirement` and `origin-demand` select lifetime optimizations; `--release-source-artifact` releases validation IR. Bundled data contains all three matrices and their hashes.

MAINTAINER BRIEF / 中文摘要

当前性能与内存结果

本报告的性能主表采用已验证的 `502e29cd` checkpoint，覆盖固定的 sum-bits 和 Raytrace Pass 3。推理中位数分别为 16.379 ms 和 6.140 s，相对预先声明的旧 Cell 基线达到 2.056× 与 2.200×。这组验收有 11 对 sum-bits 和 7 对 Raytrace 实测，均在排除预热后交替启动新进程。

同场重跑旧 Cell 得到 30.290 ms 和 12.208 s，因此同场中位数之比分别为 1.849× 和 1.988×。两个固定阈值都已通过；两者相对同场旧版都达到 2×，仍需要进一步证据。候选构建同时使用 native CPU、fat LTO、单 codegen unit，以及仅在另一个负载上训练的 PGO，这组比较反映它们与引擎改动的合计效果。

内存实验另以 `27f098a2` 为控制，使用普通 release 配对。Raytrace 峰值 RSS 从 1,856.79 降到 1,432.68 MiB，减少 22.8%。改动包括在所有消费者越过入口 epoch 后释放旧材料化行，解除无读者 Unit 完成记录对生产者的保活，以及在运行结构初始化后释放验证 JSON 和输入缓冲。推理中位数为 6.746 → 6.717 s，逐对结果有波动，本轮主要支持内存收益。

对应原始未加 marker 的 Raytrace 源码在原生 egglog 上，两次单线程峰值约为 462.375 和 450.047 MiB。两边均包含解析、初始化和推理。原生未导出图，Cell 在导出前采样。内存比约从 4.07 倍降到 3.14 倍。生成的 View 与 Applied 表占最终 Cell 图约 32.4% 的行，后续压缩必须保住查询、修复转发、锚点发布和导出的可观察行为。

本轮各 Cell 配置的逐 Run RHS、有效写入、Spine 接续、调度和最终图与冻结参考一致。第 7 节保留了跨引擎检查中的具体 TermSingle 差异。原生后端替换还需定位该差异并验证提取结果。当前性能表的覆盖范围是这两个 Pass 3；原有全套输入与结果仍作为历史证据保存在数据附件中。